DOCUMENTO DE ARQUITETURA DA SOLUÇÃO - DAS

CHECKPOINT 2: SOFTWARE ENGINEERING (arquitetura da solução) – 2CCPS

Risk Solutions

RM568438 | RM567348 | RM566746 | RM568081 | RM567328

I. CONTEXTO E CONTAINERS C4

Atores do Sistema

- **Vistoriador:** Responsável pelo preenchimento das informações técnicas e checklists em campo.
- **Gestor da Franquia:** Avalia e aprova as vistorias concluídas.
- **Auditor da Seguradora:** Acesso em modo de leitura para verificação de conformidade e relatórios.
- **Administrador de TI:** Gerencia acessos, permissões e configurações gerais do sistema.
- **Sistema Cron:** Agendador automático responsável pelo disparo de alertas e rotinas periódicas.

Ecossistema Integrado

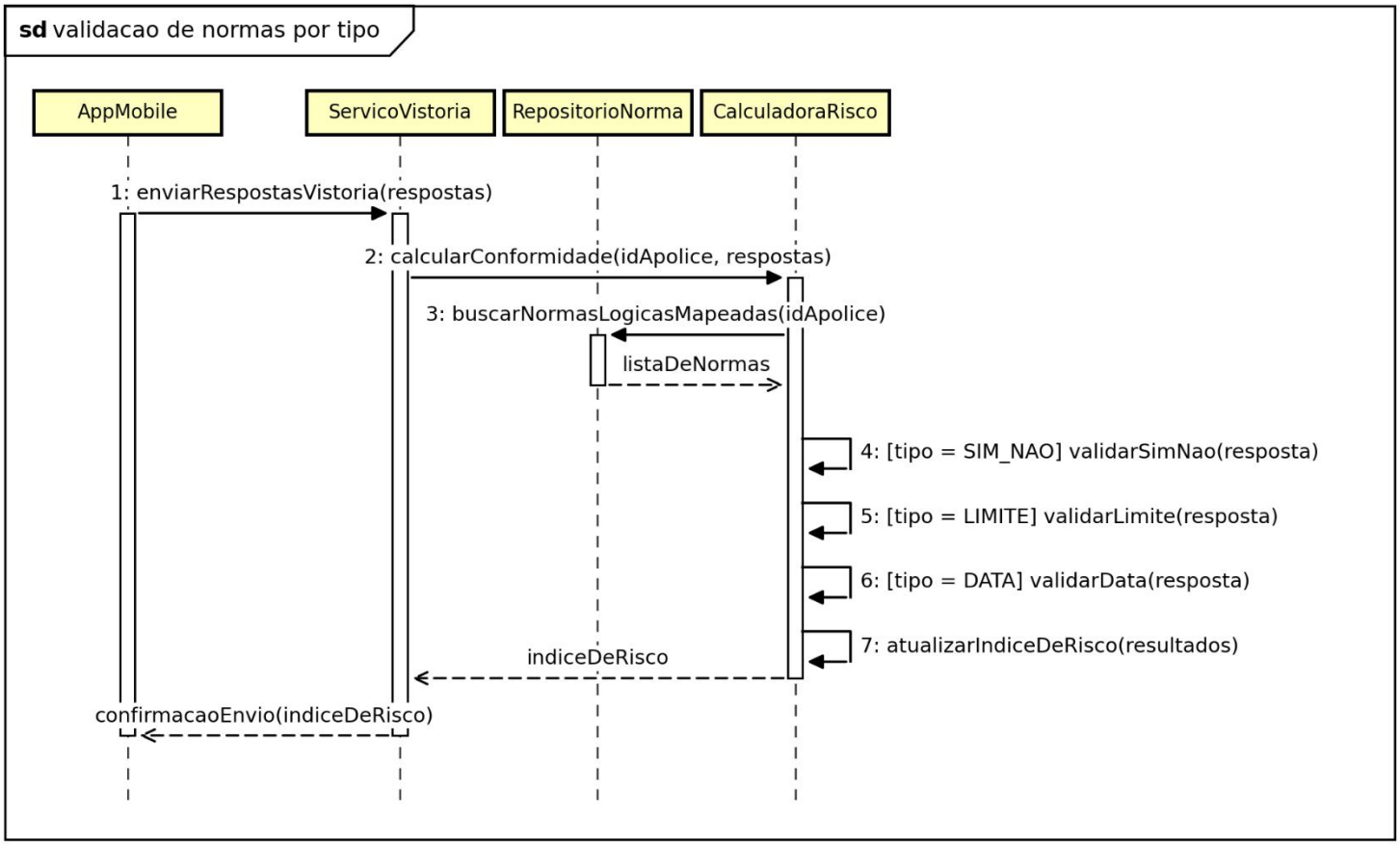
O ecossistema interno opera de forma isolada, sem dependência de APIs externas em tempo real, utilizando armazenamento e processamento local de dados de apólices em banco de dados relacional.

Mapeamento de Containers

- **Frontend:** Aplicativo Mobile / Web - Interface para captura de dados em campo pelos vistoriadores e painel de gestão para aprovações.
- **API Principal:** Node.js (Main Process) - Gerenciamento de rotas HTTP, orquestração dos serviços de vistoria e regras de cálculo de risco.
- **Worker:** Node.js (Thread Isolada) - Processamento assíncrono dedicado à geração de arquivos PDF e aplicação de hash criptográfico (assinatura).
- **Banco de Dados:** PostgreSQL - Armazenamento relacional de dados de franquias, apólices, vistorias e regras de normas.
- **File System:** Armazenamento Local - Guarda de blobs de mídia e arquivos PDF referentes aos laudos técnicos assinados.

II. DIAGRAMA DE SEQUÊNCIA (VALIDAÇÃO DE NORMAS)

O fluxo a seguir detalha o cenário crítico de processamento do checklist de vistoria e o cálculo do índice de conformidade.



1. *AppMobile* envia requisição *enviarRespostasVistoria(respostas)* para *ServicoVistoria.*

2. ServicoVistoria chama *calcularConformidade(idApolice, respostas)* na *CalculadoraRisco.*

3. *CalculadoraRisco* aciona *RepositorioNorma* via *buscarNormasLogicasMapeadas(idApolice),* que retorna a *listaDeNormas.*

4. *CalculadoraRisco* itera sobre as respostas e delega a validação via Strategy:
   - Se tipo SIM_NAO: executa *validarSimNao(resposta).*
   - Se tipo LIMITE: executa *validarLimite(resposta).*
   - Se tipo DATA: executa *validarData(resposta).*

5. *CalculadoraRisco* executa *atualizarIndiceDeRisco(resultados)* e devolve o *indiceDeRisco.*

6. *ServicoVistoria* retorna *confirmacaoEnvio(indiceDeRisco)* para o *AppMobile.*

III. MODELO DE COMPONENTES

- **Camada de Apresentação:** Controladores/Rotas HTTP que recebem os payloads JSON dos vistoriadores.
- **Camada de Serviços (Domínio/Aplicação):**

Componente	Responsabilidade
ServicoVistoria	Orquestra o recebimento, aciona o cálculo e enfileira a geração do laudo.
CalculadoraRisco	Centraliza o processamento lógico de conformidade aplicando os validadores.
ServicoAutenticacao	Emite e valida JWTs.

- **Camada de Dados (Repositórios):** Classes como RepositorioNorma que utilizam Query Builders para acessar o PostgreSQL, isolando o SQL da regra de negócio.

IV. ESTILO ARQUITETURAL

- **Escolha:** Arquitetura em Camadas (Apresentação, Regras de Negócio/Services, Dados/Repositories) desacoplada de processamento pesado via padrão Worker.

- **Justificativa:** O projeto não possui integrações externas complexas que exijam Arquitetura Hexagonal neste momento. A divisão em camadas isola o cálculo de conformidade (CalculadoraRisco) das consultas ao banco e da exposição web. A extração da geração de PDF para um Worker evita bloqueio do Event Loop do Node.js (Main Process), garantindo que a API permaneça responsiva para outros vistoriadores enquanto laudos são assinados.

- **Alternativas descartadas:** Microsserviços (excesso de custo e complexidade operacional para um escopo inicial sem orçamento financeiro).

V. PADRÃO GOF: STRATEGY

- **Problema concreto:** As normas da seguradora possuem formatos lógicos diferentes (sim/não, valor com limite, data). A validação dessas normas geraria um condicional (switch) massivo e crescente dentro da CalculadoraRisco.

- **Padrão candidato e aplicação:** Strategy. Cada tipo de norma é encapsulado em uma classe validadora que implementa a mesma interface (validar(resposta, norma)).

- **Benefício:** Uma nova regra criada pela seguradora vira uma classe isolada. A CalculadoraRisco descobre o validador em tempo de execução pelo tipo da norma, eliminando a necessidade de alterar a lógica central de cálculo.

- **Custo:** Multiplicação de classes no diretório de domínio (um arquivo por regra).

VI. ADR-001: STRATEGY NA VALIDAÇÃO DE NORMAS

- **Contexto:** O Risco 2 do projeto aponta mudanças abruptas nas regras de compliance da seguradora. A verificação de conformidade na v1 avalia regras de booleanos, numéricos e datas, com alta probabilidade de novos tipos lógicos serem exigidos no futuro.
- **Decisão:** Adotar Strategy para abstrair as regras de validação por tipo de norma na entidade CalculadoraRisco.

- **Alternativas consideradas:**
    - *Nenhum padrão (Switch-case):* Descartado por engessar a classe principal e misturar regras de data com lógicas booleanas no mesmo arquivo.
    - *Factory Method / Observer / Adapter / Decorator:* Descartados por resolverem problemas criacionais, reativos, de fronteira ou transversais, e não a mutabilidade da regra de negócio primária.

- **Consequências positivas:** Se um tipo não mapeado surgir, a engine de cálculo falha explicitamente em vez de aprovar um risco por engano. Facilita a criação de testes unitários isolados para cada regra matemática ou temporal.

- **Consequências negativas:** Exige que qualquer novo desenvolvedor compreenda a injeção/seleção dinâmica de validadores antes de adicionar um simples if.